Architecting the Research Loop: Design Choices for Hypothesis-Generating Agents

Anonymous Author(s)
Affiliation
Address
email

Abstract

Agents that generate hypotheses and iterate on them are increasingly used for automated research, but their designers face a number of architectural choices: what and how much information should be carried from one loop to the next, whether information should be shared between agents and whether agents should be able to modify their own hypothesis-generation methodology. We study these questions empirically through the construction of execution algorithms for Micro E-mini S&P 500 futures. We use tick-by-tick market data, which is known to have a high noise-to-signal ratio, making this a meaningful setting for testing agent-architecture ablations and seeing whether agents are able to discover robust execution algorithms. We fix the task and evaluation framework and vary only the architecture of the research loop, evaluating four different agent architectures that attempt to improve three base algorithms. Across all base algorithms, the best in-sample and out-of-sample results come from sharing knowledge between parallel-running agents. Second, additional deliberation during hypothesis generation did not improve performance: agents that refined a hypothesis before implementation were outperformed by a simpler loop that directly asked for a new hypothesis from the previous loop's context. Third, carrying more information between loops is not consistently beneficial: brief summaries remain competitive with full reasoning traces despite using much less context. These findings suggest that where an agent gets information from matters more than whether it reasons about its own method or retains more context.

1 Introduction

Anyone building an automated research agent faces a number of initial architectural decisions. Some questions include: what information should be carried from one research loop to the next, and how much of it? Should that information come only from the agent's own history or also from agents working in parallel on different problems? And should the agent explicitly reason about how it conducts research?

Recent work proposes a wide range of architectures for iterative agentic systems, including systems that automatically redesign agents, evolve candidate programs, reflect on their own outputs, or use multi-agent debate [Hu et al., 2025, Novikov et al., 2025, Lange et al., 2026, Zhang et al., 2026, Madaan et al., 2023, Shinn et al., 2023, Du et al., 2024]. However, these choices are often studied separately, on different tasks and under different evaluation frameworks, making their relative importance difficult to compare. Closest to our setting is Kvasiuk et al. [2026], which uses an LLM-

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

driven evolutionary process to optimize trading algorithms and compares structured evolutionary search with an unstructured coding agent. Instead of proposing another agent architecture, we ask ourselves: holding the task, evaluator and LLM model fixed, which architectural choices matter?

Our main contributions are:

1. **A leakage-controlled environment for comparing research-loop architectures**: agents can propose hypotheses and only modify the execution algorithm, but the evaluator and out-of-sample data are hidden from the agent and only accessible via an endpoint, in order to prevent reward hacking. This lets the same framework test different architecture ablations while keeping the evaluation target unchanged.
2. **An empirical comparison across four architecture families**: by comparing four families under the same protocol, we identify which design choices matter in this setting. The most useful information came from agents working in parallel on different base algorithms and sharing summaries of what worked. In contrast, agents that debated or rewrote the hypothesis-generation process did not beat a one-shot prompt for the next hypothesis, given the same prior-loop context. Also carrying more context from prior loops did not monotonically improve performance.

2 Experimental Setting

We use tick-by-tick market data for the Micro E-mini S&P 500 future (MESM6).[1] The study uses 22 trading days, containing a total 170.3 million tick records: 12 in-sample days from 2026-03-08 to 2026-03-20 and 10 out-of-sample (OOS) days from 2026-03-26 to 2026-04-06 with a five-day gap in between. All execution algorithms act on the same fixed oracle signal: once per second, the strategy compares the current trade price with the trade price 30 seconds later, adds Gaussian noise, and suggests a one-contract buy or sell signal when the noisy future return exceeds a fixed threshold. The agent's task is then the optimal-execution problem [Almgren and Chriss, 2001]: deciding whether and how to submit each order.

Figure 1 summarizes the experimental pipeline. Each agent is defined by a markdown file in the `.claude/agents` directory and is given a budget of eight research iterations. During those iterations, agents can propose hypotheses and write candidate execution algorithms, but the evaluator, data split, and OOS data remain outside the agent loop. We use the open-source NautilusTrader package as the backtesting engine for our evaluator.[2] Agents only receive training metrics during their research loops. After eight iterations, the best execution algorithm on the training set is evaluated once on OOS data. We select the algorithm by training-window Sharpe ratio, and report P&L, implementation shortfall, and hit rate alongside it.

2.1 Agent architectures

We compare four research-loop architecture families. All agents use Claude Opus 4.7, have web search enabled, and receive relevant prior literature as context.

Per-iteration context ablation. This family keeps the research loop unchanged and varies only how much previous-loop context the agent receives. In *metrics-only*, the agent receives only the raw metrics from prior attempts. In *full-trace*, it sees the complete reasoning trace, while *brief-summary* contains a short summary of that trace. This isolates whether more context improves performance.

***PC* (proposer–criticizer debate)** [Du et al., 2024]. *PC* adds deliberation before implementation. A proposer states a hypothesis and a criticizer challenges it before code is written. They iterate until agreement or three debate rounds is reached, and the final hypothesis is implemented and evaluated. This tests whether the agent should spend more effort in the hypothesis generation process.

[1] The data comes from Databento's GLBX.MDP3 CME Globex MDP 3.0 dataset: https://databento.com/datasets/GLBX.MDP3.

[2] https://nautilustrader.io

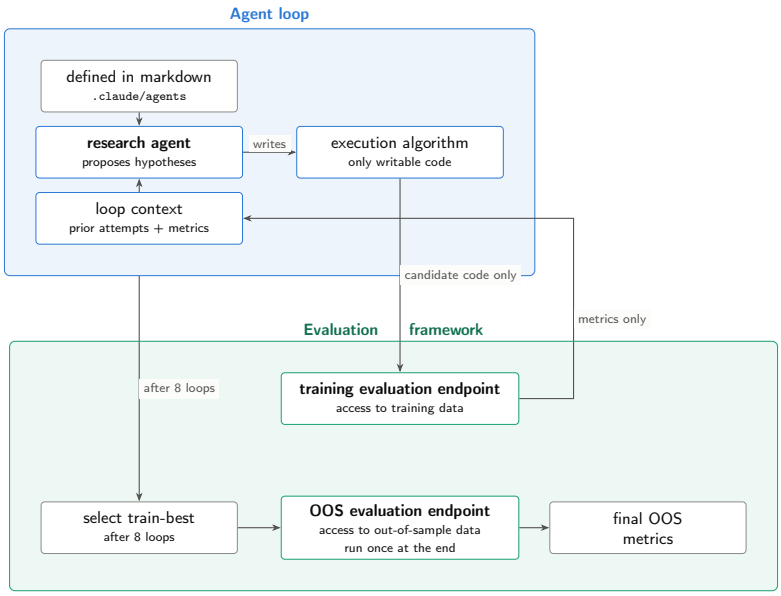


Figure 1: Evaluation pipeline. The agent loop can modify only candidate execution algorithms. The evaluation framework owns the training and OOS data; endpoints return metrics without exposing the data or evaluator internals. OOS evaluation is run once after all loops are complete.

***SIP* (self-rewriting prompt)** [Spiess et al., 2025, Yin et al., 2025]. *SIP* is a meta-agent that lets the system modify the hypothesis-generation process itself. After each loop, a critic reads the researcher's reasoning trace, identifies one failure mode in how the hypothesis was generated, and rewrites the hypothesis-generation prompt. Prompt changes are accepted only when they improve performance on the training set. This tests whether the agent can improve its own hypothesis-generation process.

***Island* (population-based migration)** [Tanese, 1989, Romera-Paredes et al., 2024]. *Island* adds cross-agent information sharing. We run three agents in parallel, one for each base algorithm. Each agent improves its own base algorithm for two iterations, after which a migration report summarizes the best results and short lessons from all three agents. The report is shared with every agent, but agents do not see each other's code. This tests whether agents benefit from shared information compared to working in isolation, as the other three architecture families do.

2.2 Starting point: three base algorithms

Before comparing research-loop architectures, we first used a separate agent to generate three independent execution algorithms. That agent was asked to improve over a simple market-order baseline. The resulting algorithms become the starting points for all agents: each architecture is tasked to improve the bases. Using distinct base strategies makes the results less dependent on the starting algorithm and lets us compare whether each architecture improves across different search problems.

PTG (position-tier-gate) uses portfolio state as its only signal. It allows the first opening order in a direction, but skips additional opening orders once the current net position reaches a fixed cap. Reduce-only and closing orders still execute, so PTG mainly controls how much exposure the oracle signal is allowed to build.

AFG (aggressor-flow-gate) uses recent trades to estimate short-term aggressor flow. Before submitting an opening order, it checks whether recent buyer- or seller-initiated trades are moving against the proposed direction. If the recent flow is adverse, it skips the opening order, otherwise it submits it. AFG therefore acts as a microstructure filter around the oracle signal.

VRS (vol-regime-sizer) uses volatility regimes to adjust its execution strategy. It submits orders with a probability that falls during a high-vol regime, i.e. when short-horizon volatility is high relative to longer-horizon volatility and increases during a low-vol regime.

We run six total agents (three per-iteration context variants, *PC*, *SIP*, and *Island*) on three base algorithms for eight research iterations, giving $6 \times 3 \times 8 = 144$ total iterations. Token spend is recorded per iteration and used for the token cost analysis.

3 Results

3.1 Context ablation: more context is not always better

We begin with the context ablation because it isolates the simplest question: whether giving the agent more prior-loop information improves the search.

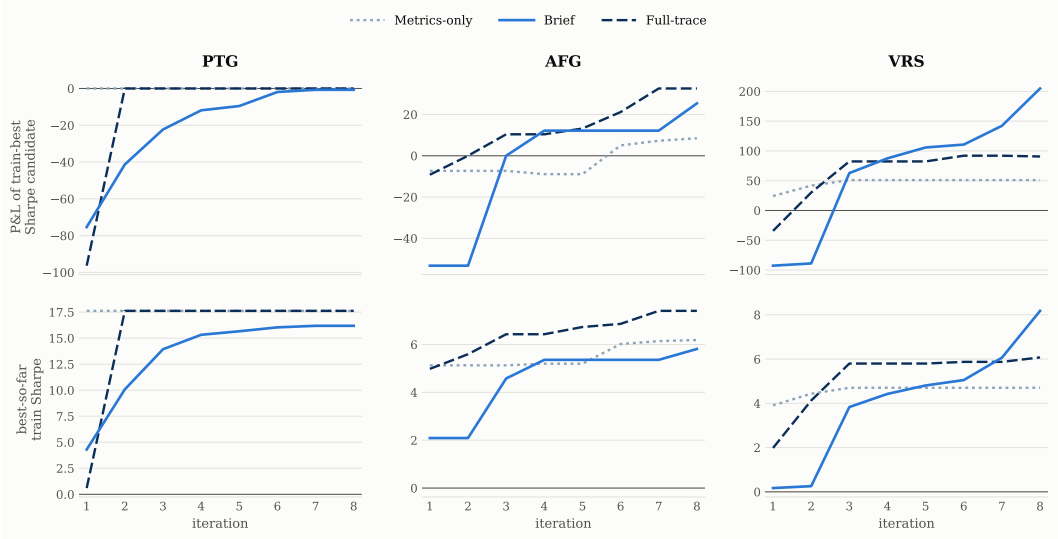


Figure 2: Training performance for the per-iteration context ablation. The top row is training P&L improvement (% vs own base) of the best-Sharpe candidate so far; the bottom row is the best training Sharpe reached so far.

Figure 2 does not show a clear monotonic relationship between context size and training performance. *Full-trace* improves the AFG base strategy the most across both metrics, but is outperformed by *brief-summary* on VRS. *Metrics-only* is weaker than both on VRS, but remains competitive on AFG. None of the three improves PTG on training P&L: *metrics-only* and *full-trace* finish at 0% P&L improvement compared to the base, and *brief-summary* slightly underperforms.

Table 1: Average prior-loop context size in the context ablation.

Condition	PTG	AFG	VRS	Mean
Metrics-only	317	317	319	318
Brief-summary	12,182	10,468	10,778	11,143
Full-trace	20,916	78,912	86,189	62,006

Table 1 confirms that *full-trace* passes far more text than *brief-summary*, and both are much larger than *metrics-only*. Yet the larger input does not translate into consistently better training performance. The useful information appears to be compressible, carrying the whole reasoning trace forward is not automatically better. *Full-trace* never finishes training behind *metrics-only*, so we take *brief-summary* and *full-trace* as the baselines in the comparison to the other architectures.

3.2 Architecture comparison: information sharing beats self-deliberation

Next, we compare the four architecture families to test whether self-deliberation or information sharing helps, and whether the training results also hold out-of-sample.

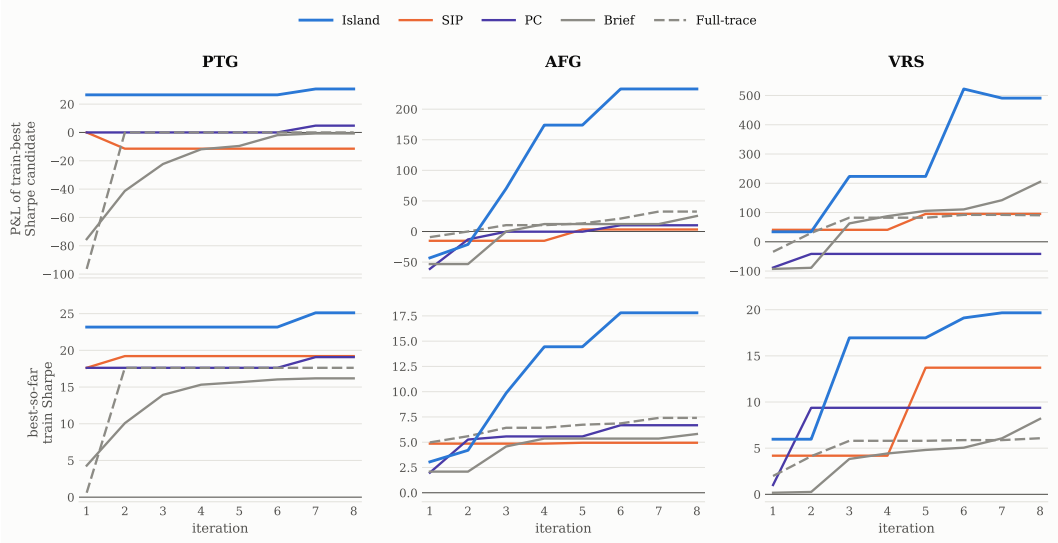


Figure 3: Training performance for the four architecture families comparison. The top row shows the training P&L improvement (% vs own base) of the best-performing candidate so far; the bottom row shows the best training Sharpe reached so far.

Figure 3 shows a clear training result: *Island* outperforms every other architecture on all three bases, offering large improvement particularly on the AFG and VRS bases and is the only method that clearly improves PTG. *PC* and *SIP* do not separate from the context-only loops; it is not clear from the training curves whether self-deliberation beats one-shotting the next hypothesis (context-only). Furthermore, we cannot judge whether *Island* genuinely improves the base strategies or manages to overfit the training set. We therefore consider out-of-sample results in Table 2.

Table 2: OOS realized P&L improvement (% vs own base) relative to each base algorithm (%). Ranks within each base are given in parentheses (1 = best). The final two columns compare the mean rank each architecture reached on the training window with its mean OOS rank.

Architecture	AFG	PTG	VRS	IS rank	OOS rank
***Island* (migration)**	**+99.9** (1)	**+24.5** (1)	**+178.3** (1)	**1.00**	**1.00**
Per-iter, *brief-summary*	+7.2 (3)	−0.5 (4)	+33.0 (3)	3.67	3.33
Per-iter, *full-trace*	+11.6 (2)	0.0 (2.5)	+14.3 (6)	3.83	3.50
Per-iter, *metrics-only*	−9.1 (4)	0.0 (2.5)	+15.5 (5)	5.17	3.83
SIP (self-rewriting prompt)	−17.2 (5)	−4.0 (6)	+71.3 (2)	4.33	4.33
PC (proposer–criticizer)	−19.3 (6)	−1.2 (5)	+24.8 (4)	3.00	5.00

OOS evaluation gives a clearer ordering. *Island* ranks first on all three bases, with gains of $+24.5\%$ on PTG, $+99.9\%$ on AFG, and $+178.3\%$ on VRS. Only *Island* improved PTG versus its own base; the others are at best unchanged.

The negative result is just as important. *PC* and *SIP* both add extra deliberation to the hypothesis-generation process, but neither beats the simpler context-only baselines on mean OOS rank, which used a one-shot prompt to generate the next hypothesis. Particularly, their improvements on AFG did not hold out-of-sample, producing worse results than base strategy and only improving the VRS base strategy. *PC* finishes last overall in terms of OOS rank, and *SIP*'s strong VRS result is offset by regressions on AFG and PTG.

3.3 Token cost analysis

One possible explanation for *Island*'s advantage is that migration simply gives agents more text to read and allows for increased token usage. To check this, we record the token usage by the LLM for each loop, counting total input, output, and cache tokens.

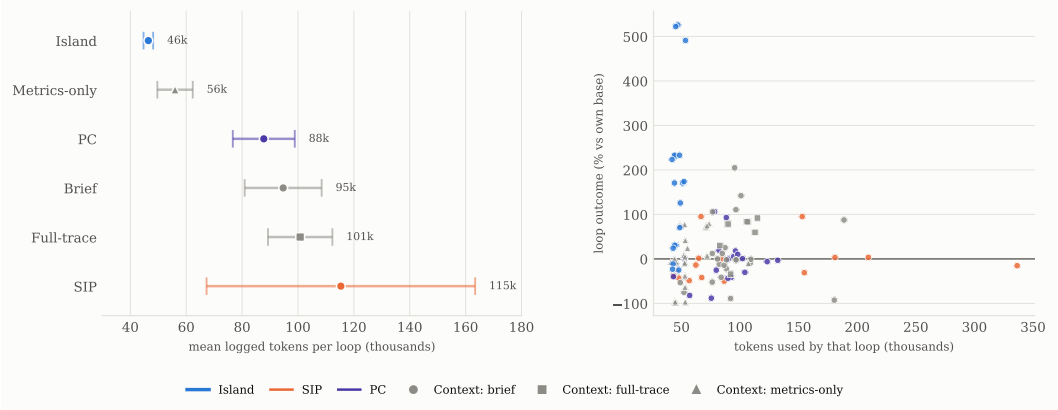


Figure 4: Token cost analysis. **Left:** mean logged tokens per loop with 95% CIs. **Right:** token spend for each logged loop against that loop's outcome.

Figure 4 shows that *Island* is not winning by spending more tokens. Among the active architectures, it has the lowest mean logged token cost per loop (46.4k). *PC* and *SIP*, which add extra deliberation before implementation, are more expensive. In particular *SIP*, whose token cost was highly variable due to its ability to change its own research methodology.

Spending more within a loop also does not predict a better loop outcome ($\rho = -0.03$, $p = 0.79$). The advantage lies in the source of the information. Ideas discovered independently, and already shown to help on a related problem, were more useful than a longer record of a single agent's own search.

4 Responsible Use

This work is an offline study on historical market data. We do not deploy a trading agent, execute trades, or provide investment advice. As such, we expect no direct societal impact from the study itself, though any future deployment of similar systems would require separate review.

5 Conclusion

Holding the task, model, evaluator, and loop budget fixed, *Island* migration was the strongest architecture in this setting. It ranked first on all three OOS evaluations while using the lowest measured token cost of any active architecture. In contrast, *SIP* and *PC* added extra deliberation to hypothesis generation but did not outperform simpler context-only baselines.

The main lesson is that routing information across agents was more useful than giving a single agent more context or more opportunities to reason about its own hypotheses. We note that a limitation of this study is that the results only use one LLM model, eight iteration loops, and three bases, which should be expanded in future work. Since the strongest signal in our experiment came from independent agents sharing useful ideas, LLM model diversity may be another way to increase the diversity of the search. The negative results for *SIP* and *PC* also suggest that, in this setting, directly proposing and testing hypotheses was more useful than spending extra effort on the hypothesis generation process itself. Finally, *Island* migration deserves a more detailed study on how much information should be transferred, in what form, and at what stages of the research loop the migration step should occur.

References

S. Hu, C. Lu, and J. Clune. Automated design of agentic systems. In *ICLR*, 2025. arXiv:2408.08435.

A. Novikov et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. Technical report, Google DeepMind, 2025. arXiv:2506.13131.

R. T. Lange, Y. Imajuku, and E. Cetin. ShinkaEvolve: Towards open-ended and sample-efficient program evolution. In *ICLR*, 2026. arXiv:2509.19349.

J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. In *ICLR*, 2026. arXiv:2505.22954.

Y. Kvasiuk, T. Li, O. Colegrove, and M. Münchmeyer. MadEvolve: Evolutionary optimization of trading systems with large language models. *arXiv preprint arXiv:2605.23007*, 2026.

A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. In *NeurIPS*, 2023.

N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning*, pages 11733–11763. PMLR, 2024.

C. Spiess, M. Vaziri, L. Mandel, and M. Hirzel. AutoPDL: Automatic prompt optimization for LLM agents. In *Proceedings of the Fourth International Conference on Automated Machine Learning*, pages 13/1–20. PMLR, 2025.

X. Yin, X. Wang, L. Pan, L. Lin, X. Wan, and W. Y. Wang. Gödel Agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 27890–27913, 2025.

B. Romera-Paredes et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

R. Tanese. *Distributed Genetic Algorithms for Function Optimization*. PhD thesis, University of Michigan, 1989.

R. Almgren and N. Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3:5–39, 2001.